

An Investigation into using LLMs for modelling outbreaks of Ebola

Introduction

This eight-week research project focused on developing an understanding of the ability of Large Language Models (LLMs) to produce compartmental model code. Models for disease outbreaks of Ebola were of particular interest. Thus, we investigated the suitability of code generated for simple compartmental models and the relevance of changes made to the code to tailor the model for an Ebola outbreak. We then attached academic literature to the prompts to determine the level of consistency the generated model code had with the topics discussed in the papers.

The project has been completed in collaboration with the UKHSA, a government organisation which aims to prevent and reduce harm to people from various health threats (GOV.UK, 2021). The UKHSA make use of predictions from mathematical models to inform decisions. The rest of this case study discusses the suitability of the implementation of some LLMs to this aim.

Motivation

In the face of disease outbreaks, reliable predictions for future dynamics of the spread of infection are key to making informed decisions. This means healthcare systems are better equipped to cope with the strain when this happens, and resources aren't wasted when and where they are less required (Gao et al., 2025).

Predictions produced by different models for the spread of infection can be weighted together to produce an ensemble. It has been found that predictions from these multi-model ensembles are often more reliable and accurate (Shandross et al., 2026).

We have aimed to investigate whether we can depend on the use of LLMs to speed up the ensemble generation process. We have been interested in the ability of different LLMs to generate compartmental model code for disease spread models found in academic literature. If this process is found to be

successful, the code for single models can be generated quicker, and hence aggregated to produce an ensemble more efficiently.

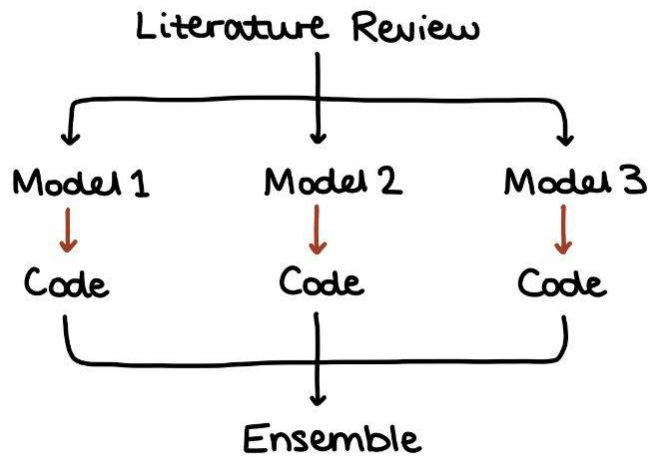
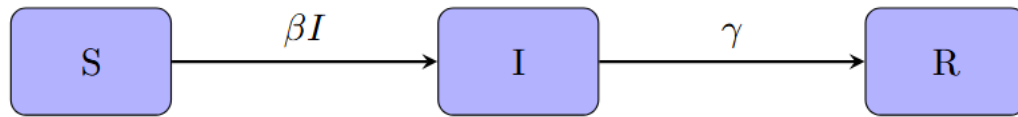


Figure 1: Producing an Ensemble

Model Structure

Compartmental models are often used in epidemiology. The population is split into distinct groups, each described by an individual's current disease status. The movement of individuals between compartments is modelled by a system of ordinary differential equations (ODEs). The solutions to these equations, found via numerical methods, provide the number of individuals in each compartment at each time step. These disease trajectories are used for predictions which then help to construct informed decisions. (Keeling and Rohani, 2008)



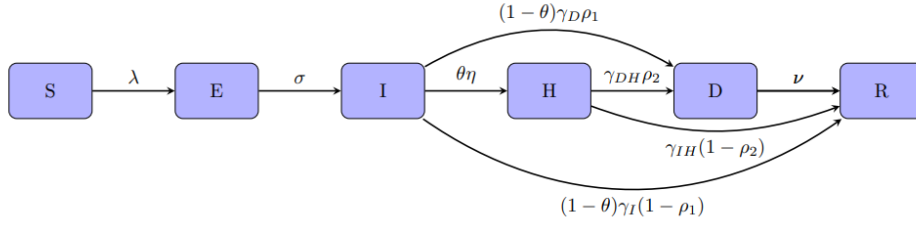
$$\begin{aligned}\frac{dS}{dt} &= -\beta SI \\ \frac{dI}{dt} &= \beta SI - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

Figure 2: State transition diagram and ODEs for SIR model

The SIR model splits the population into 3 compartments and is used in epidemiology for modelling the disease dynamics in this simple case. Individuals in the population are categorised as susceptible (able to be infected by the pathogen), infected or recovered (cleared the pathogen and received immunity).

To construct a compartmental model specified to Ebola we need to consider the key epidemiological features of the disease (Legrand et al., 2006).

Ebola is an infectious flu-like virus spread through direct transmission. An infected individual is not infectious immediately. We represent this incubation period, estimated at 2-21days, in the model using an exposed compartment (E). Isolation in hospital is the usual treatment for Ebola, where individuals are kept hydrated and quarantined to reduce transmission. Recently deceased individuals contribute to transmission, and removal from this group is modelled by a rate of safe burial. Splitting the infectious population into the infected in community, infected in hospital, and infected deceased allows us to consider the differing transmission rates between these groups (WHO, 2025).



$$\lambda = \frac{\beta_I I + \beta_H H + \beta_D D}{N}$$

$$\frac{dS}{dt} = -\lambda S$$

$$\frac{dE}{dt} = \lambda S - \sigma E$$

$$\frac{dI}{dt} = \sigma E - \left[\theta \eta + (1 - \theta) [\gamma_I (1 - \rho_1) + \gamma_D \rho_1] \right] I$$

$$\frac{dH}{dt} = \theta \eta I - \left[\gamma_{IH} (1 - \rho_2) + \gamma_{DH} \rho_2 \right] H$$

$$\frac{dD}{dt} = (1 - \theta) \gamma_D \rho_1 I + \gamma_{DH} \rho_2 H - \nu D$$

$$\frac{dR}{dt} = (1 - \theta) \gamma_I (1 - \rho_1) I + \gamma_{IH} (1 - \rho_2) H + \nu D$$

Figure 3: State transition diagram and equations for SEIHDR model

Code Representation of Compartmental Models

The disease dynamics are modelled by a complicated system of ODEs, and the solution to this involves numerical approximations. In the code generation portion of this project, we have used Odin and PyGOM in R and Python respectively to find the solutions to these equations.

Odin is a Domain Specific Language (DSL) in R for describing and solving ODEs (GitHub, 2026). PyGOM is a package in Python for building and solving ODEs, with a focus on epidemiological models (Tye et al., 2018).

```

1 library(odin)
2 library(ggplot2)
3
4 # Compiling the model
5 sir_model <- odin({
6   # Differential equations
7   deriv(S) <- - beta * S * I / N
8   deriv(I) <- beta * S * I / N - gamma * I
9   deriv(R) <- gamma * I
10
11   # Initial conditions
12   initial(S) <- S_ini
13   initial(I) <- I_ini
14   initial(R) <- R_ini
15
16   # Total population
17   N <- S + I + R
18
19   # User-defined parameters
20   beta <- user()
21   gamma <- user()
22
23   S_ini <- user()
24   I_ini <- user()
25   R_ini <- user()
26 })
27
28
29 states = ['S', 'I', 'R']
30 params = ['beta', 'gamma']
31 transitions = [
32   Transition(
33     origin='S',
34     equation='-beta*S*I',
35     transition_type=TransitionType.ODE
36   ),
37   Transition(
38     origin='I',
39     equation='beta*S*I - gamma*I',
40     transition_type=TransitionType.ODE
41   ),
42   Transition(
43     origin='R',
44     equation='gamma*I',
45     transition_type=TransitionType.ODE
46   )
47 ]
48 sir = DeterministicOde(
49   state=states,
50   param=params,
51   ode=transitions
52 )

```

Figure 4: SIR code using Odin (left) and PyGOM (right)

Stan is a probabilistic programming language that has been used in this project to fit Bayesian compartmental models to data (Grinsztajn, 2021).

Fitting Compartmental Models to Data

Fitting models to data helps to provide reasonable estimates for parameters and decide to what extent they seem to produce accurate realisations of real-world data (Martcheva, 2015), (Hoang and Ehrhardt, 2025).

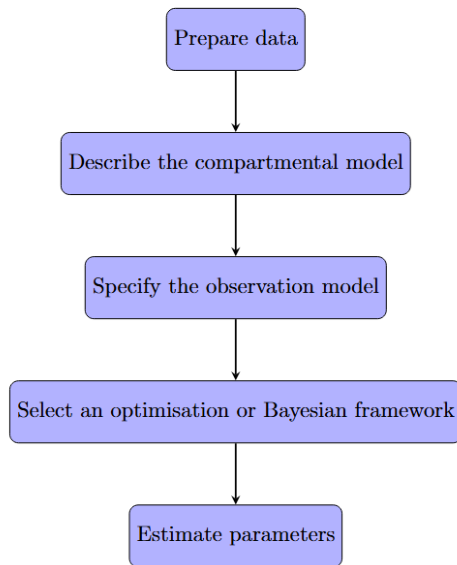


Figure 5: Fitting compartmental models to data framework

The compartmental model describes the true disease systems, but real-world data is often noisy with under-reporting. Hence, we introduce an observation model to capture more uncertainty in the model (Domenech de Celles and Kramer, 2026). A Negative-Binomial observation model is often preferable to a Poisson model in this context, since the extra dispersion parameter helps to model situations with substantial overdispersion.

In the Frequentist framework, an objective function is optimised to find a single best fit of parameters. This may involve minimising the sum-of-squares error in Least Squares Estimation (LSE) or maximising the likelihood in maximum Likelihood Estimation (MLE).

In the Bayesian framework, the whole posterior distribution is characterised, so multiple possible parameter estimates are provided in proportion to how well each fits to the data (Pourzanjani, 2018). Prior distributions for parameters are specified as well as the observation model, and the posterior distribution is found using Bayes' Theorem.

Resources used

This project intended to compare multiple LLMs and interfaces they are accessed through.

We have made use of 2 Copilot interfaces: GitHub Copilot through Visual Studio Code (using an education account) and the Copilot chat function through Microsoft 365.

The Microsoft 365 Copilot gives the option for different GPT models as well as an auto function where the Copilot selects a model and length of time to think itself. Originally, the GPT models available were 5.2 or 5.5 with Quick Response or Think Deeper functions in both. As the project progressed, this updated to only 5.5 Quick Response and 5.6 Think Deeper.

The chat function made us of in Visual Studio (VS) Code selected a model of its own choice, providing the model it has used at the end of each response. This was generally MAI-Code-1-Flash, Claude Haiku 4.5 or GPT 5.4 mini. The VS Code interface was particularly useful for this project's aim. The integrated chat panel and workspace made code generation and debugging more convenient. Copying code between other AI interfaces and the desired Integrated Development Environment (IDE) was much more cumbersome.

Additionally, some basic prompting was provided to an LLaMA vllm to investigate the code generated by this open-source AI. The prompting was completed through a Pi assistant, and the LLM used was the Qwen 3.6 35B A3B FP8 model. We installed the pi-hermes-memory package to allow the LLM to learn from previous prompting and outputs.

This project made use of the WHO GitHub repository for the current outbreak of Ebola (2026 Bundibugyo Ebola outbreak), (INRB-UMIE, 2026).

Results and Discussion

Initially, prompts were provided for generating compartmental models for Ebola without attaching literature. Here, we aimed to compare success across programming languages and investigate the ability of different LLMs to incorporate other concepts into the code.

Generic Compartmental Models

Example Prompt:

Code up a compartmental model for the disease spread of Ebola using R/Python/Odin/PyGOM

All 3 interfaces successfully generated code for SIR models. The VS Code MAI-Code-1-Flash LLM needed the prompt to specify which PyGOM classes to use (SimulateODE etc.), otherwise the code would call outdated ones. Further PyGOM code through the VS Code interface was written more effectively due to the LLM access to the existing workspace with correctly formed code blocks.

The Isambard vllm initially uses strange syntax, not in line with current Odin or PyGOM documentation. This LLM learns from prompts describing the issues and searches current documentation for the format desired for the code to be written in. The code subsequently generated consistently then reproduces the desired format.

```
8 parameters {
9   beta : # transmission rate
10  gamma : # recovery rate
11  N : # total population
12 * }
13
14 state {
15   S : # susceptible
16   I : # infected
17   R : # recovered
18 * }
19
20 init {
21   S <- N * 0.999
22   I <- N * 0.001
23   R <- 0
24 * }
25
26 derivatives {
27   dS <- -beta * S * I / N
28   dI <- beta * S * I / N - gamma * I
29   dR <- gamma * I
30 * }
31
32 include "odin_base.odin"

sir_odin <- function(beta = 0.3,
                     gamma = 0.1,
                     S0 = 999000,
                     I0 = 1000,
                     t = 0:200) {

  model <- odin({
    deriv(S) <- -beta * S * I / N
    deriv(I) <- beta * S * I / N - gamma * I
    deriv(R) <- gamma * I

    initial(S) <- S0
    initial(I) <- I0
    initial(R) <- 0

    N <- S0 + I0

    beta <- user(0.3)
    gamma <- user(0.1)
    S0 <- user(999000)
    I0 <- user(1000)
  })

  instance <- model$new(beta = beta, gamma = gamma, S0 = S0, I0 = I0)
  as.data.frame(instance$run(t = t))
}
```

Figure 6: Code using Odin produced by Isambard vllm, initial incorrect use (left) and corrected use (right)

Coding Models for Outbreaks of Ebola

Example prompt:

Code an SIR model using R/Python/Odin/PyGOM

-> Extend this SIR model to model key aspects of an Ebola outbreak

For the Copilot in VS Code (using MAI-Code-1-Flash) to code a model tailored to Ebola, prompts had to specifically mention the disease features to be included. The GPT models through Microsoft 365 tended to suggest full SEIHFR models, but an SEIRD model was once suggested as a “minimal teaching model”. This simplified model may have been given due to excessive previous prompting. The Lsambard vllm required guidance for correct parameterisation of the full model.

This branch of prompting indicates the necessity of prompts in certain LLM interfaces to specify the key epidemiological features for inclusion, or the parameterisation desired for the system.

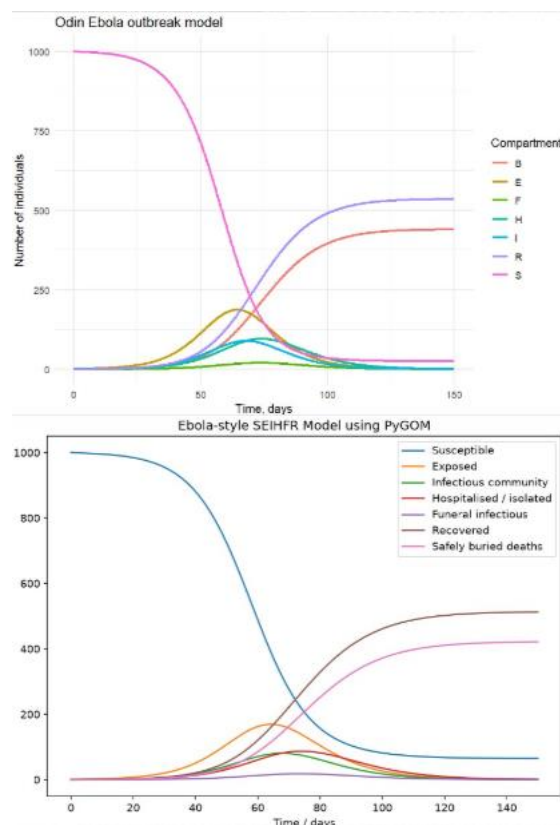


Figure 7: Trajectories from code generated using Odin (above) and PyGOM (below)

Fitting Compartmental Models to Data

The ability of LLMs to fit their generated models to data was assessed using Copilot LLMs through Microsoft 365.

How should data be provided?

The LLMs couldn't directly retrieve CSV contents from the link to the GitHub repository. The models were able to process and analyse data from copied tables of data and attached downloaded CSV files. It was useful to explicitly describe how zero-values and cumulative data should be considered in prompts.

What were the differences between LLMs?

The GPT 5.6 Think Deeper spent a significantly longer amount of time to produce output for fitting models to data than alternative LLMs. The GPT 5.5 Quick Response model included all aspects prompted for and responded faster than the 5.6 model. However, this sometimes came at the expense of simplifying the specified coding methods.

The GPT models were more likely to follow through on the recommendations provided in discussion sections of output. The auto model however was seen to highlight this a Negative Binomial model as the most suitable but use a Poisson model in code for simplicity.

Can the models fit compartmental models to data using both Bayesian and Frequentist methods?

The LLMs provided code for both frameworks. Without any specific prompting, all models use a Frequentist framework. Specifically, for the auto model this was often by maximum likelihood estimation but was also frequently by least-squares. The GPT models mainly made use of MLE. The LLMs could produce code to find RMSE and coverage for many different compartmental models and fitting methods.

Output considering Frequentist methods was frequently well specified. Different optimisation algorithms were used for fitting the models using MLE: L-BFGS-B, BFGS, Nelder-Mead.

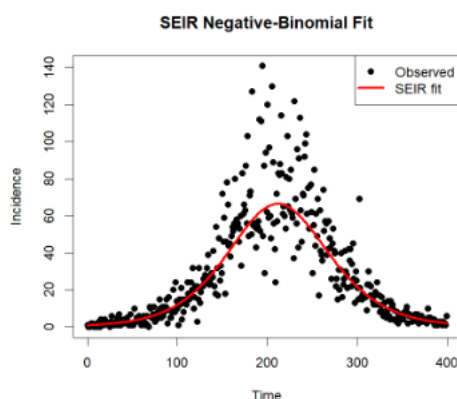


Figure 8: SEIR model fitted to simulated data using MLE after correcting for reasonable initial conditions and population size

More syntax issues were found in Bayesian coded solutions. Code generated was often incompatible with current syntax for stan code. Specifying the rstan version in use allowed the code provided to be in line with current syntax.

The prior distributions were often well chosen, with positive distributions given for rate parameters and beta distributions for probabilities or proportions. However, the parameters chosen for these priors often lead to trajectories prone to explosion or unreasonable compared to the disease trajectory in the data. Care should be taken to ensure priors aren't too uninformative.

Comments:

The ease of code generation and debugging was improved when prompts specify the framework for fitting, programming tool versions and observation model required.

Despite this seeming success, models often fit too many parameters than can be identifiable – leading to poor scaling of fits. Models were also seen to fit too few parameters to successfully describe the disease trajectory.

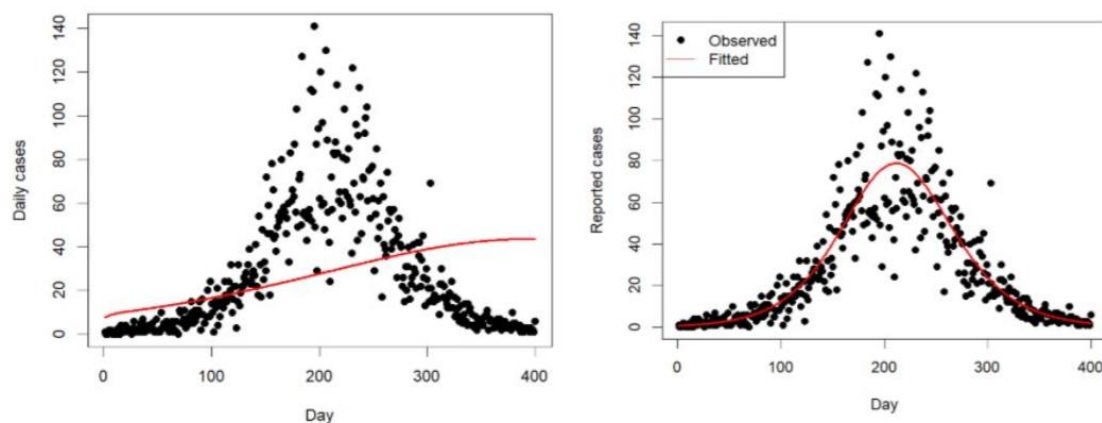


Figure 9: SEIHFR model fitted to simulated data, initially only estimating 3 parameters (left), plot after prompted to estimate all transmission parameters and the reporting rate (right)

Generating Code with Attached Literature

The LLMs found through VS Code and Microsoft 365 were provided with prompts to determine whether they could successfully generate code consistent with the contents of attached literature.

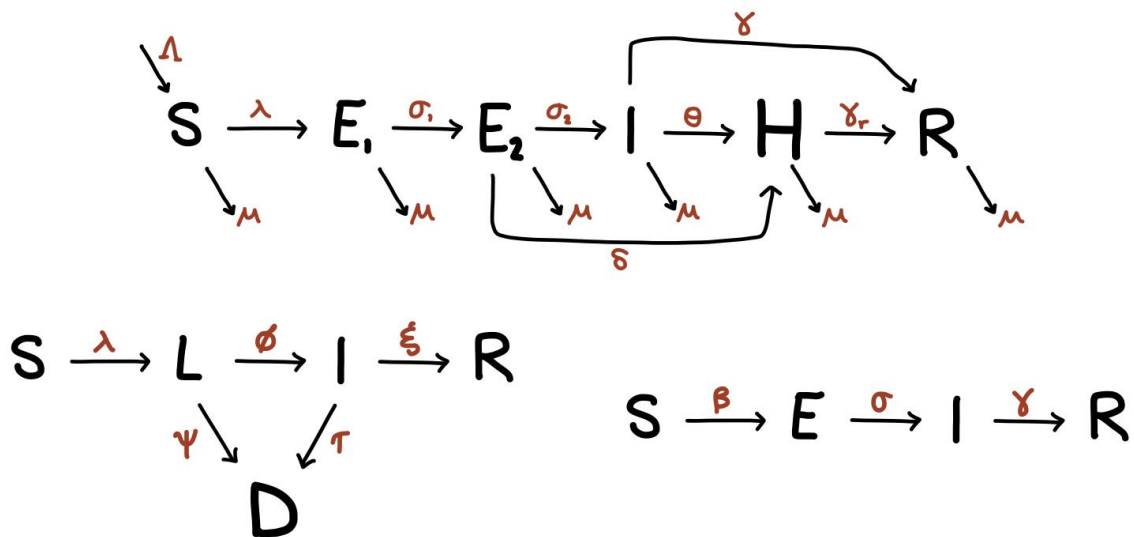


Figure 10: Other example models for Ebola outbreaks

Example prompt:

Code up a compartmental model for the disease spread of Ebola using Odin (with a pdf or link to the paper attached)

Copilot models through Microsoft 365:

Prompts were given with attached literature containing varying compartment splits and areas of interest. The LLMs here followed the correct model structure and mentioned the key aspects of paper. The responses from the Think Deeper models had the most detail provided regarding extensions detailed in the paper. The Quick Response models were more likely to only provide code necessary for modelling the compartmental model.

Copilot models through VS Code:

The Copilot models accessed through VS Code required much more explicit prompting to generate code even vaguely consistent with the attached literature. These models had a preference to build models based only on the current workspace. Additional prompts could be given to lead to the LLM to access the literature and generate more consistent code. This was implemented in future prompts.

An unsuccessful prompt, for Claude Haiku 4.5, was “Generate code using Odin to model an outbreak of Ebola, using the compartmental model given in this paper. Match the exact notation from the paper.”. This generated code for an SEIHRDB model disregarding the paper’s contents. Additionally, the content was described as taken from a hallucinated paper. The link provided by the LLM for this literature was for an unrelated paper.

Since, in comparison, even the GPT 5.2 models on the Microsoft 365 Copilot were able to complete these prompts, it suggests that either it is the mini version that is limiting this response, or restrictions from the VS Code interface.

Comments provided throughout, e.g. “Response cleared due to possible match to public code, retrying with modified prompt.”, indicates that there are barriers in place in this interface to limit copying of online code.

Fitting Compartmental Models from Academic Literature to Data

Again, the prompting here made use of the GPT models through Microsoft 365. Even after a range of prompts to adjust the fit of a model, the overall code provided still follows the structure and discussions of the attached literature.

Method of parameter estimation was not automatically chosen to be the same as used in the literature. Here, the desired method to be used was stated in prompts, but to match the framework of the literature, explicit prompting is required.

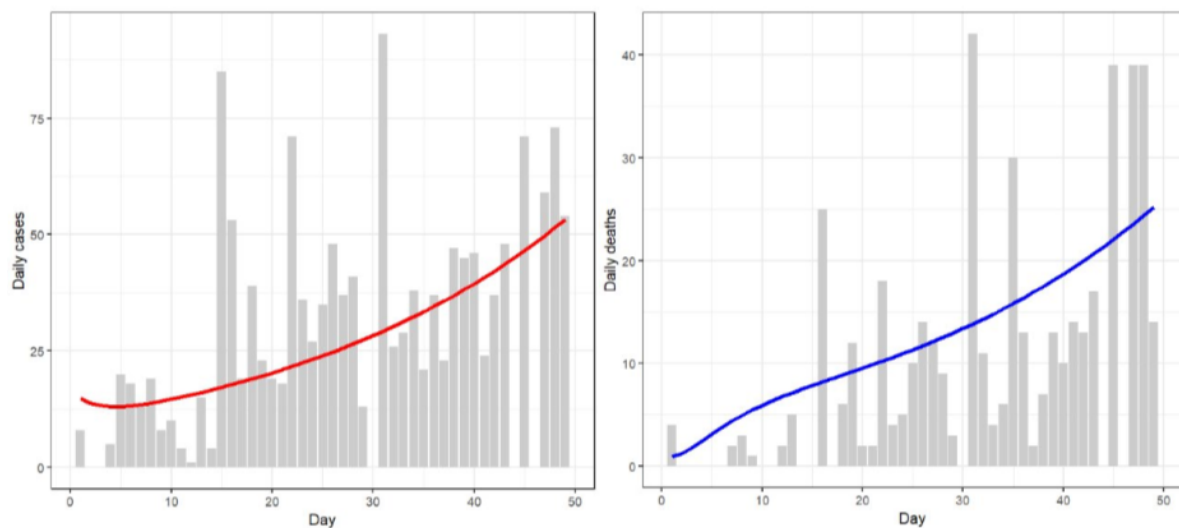


Figure 11: Fitted cases and deaths for SEIRF model (D'Silva and Eisenberg, 2017) by MLE with Poisson observation model

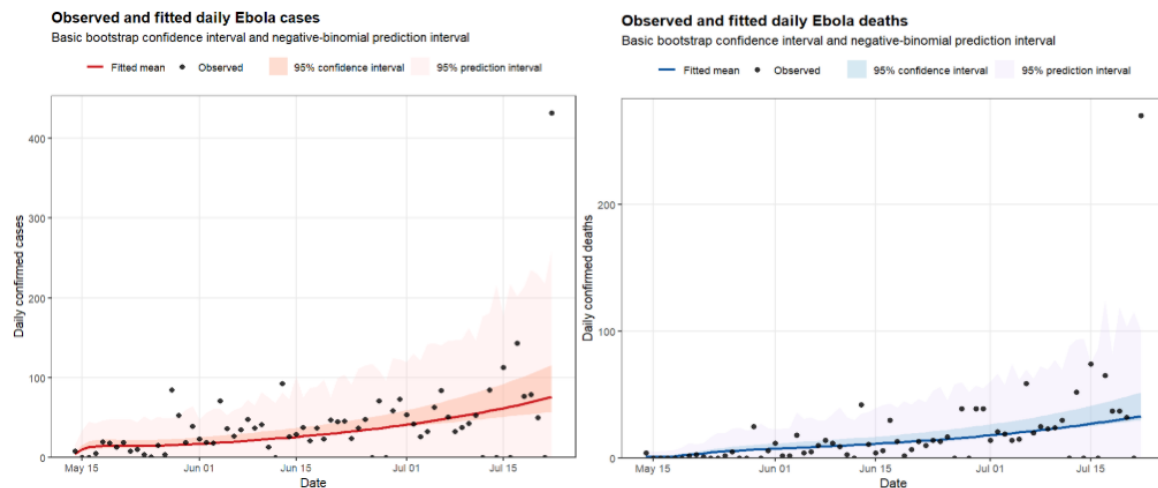


Figure 12: Bootstrap confidence intervals and prediction intervals using Negative-Binomial observation model for fitted SEIRF model (D'Silva and Eisenberg, 2017)

Challenges and Conclusions

This project aimed to investigate the ability of different LLMs to generate compartmental model code for outbreaks of Ebola. The generation of generic compartmental model code was performed correctly by all 3 AI interfaces used. Expansion of models to consider the dynamics of an Ebola outbreak was considered well but simplified model code was often provided despite acknowledgement of other epidemiological features. The Microsoft 365 Copilot models executed this aim most successfully. Approaches to producing code in line with academic literature was more varied, most likely due to interface specific restrictions.

The responses through VS Code and the Isambard vllm were more consistent than through Microsoft 365, due to the influence of code in the current workspace and the memory package respectively.

The comments and warnings provided during prompting on the VS Code interface indicate barriers limiting the output of information matching online content and conveying too much information about sensitive topics.

Comments include:

- "Response cleared due to possible match to public code, retrying with modified prompt."
- "Not a verbatim copy of a public template"
- Invalid prompt: we've limited access to this content for safety reasons. This type of information may be used to benefit or to harm people.

These barriers limit the applicability of generating compartmental model code using academic literature at a large scale.

Further analysis of output from the Copilot through VS Code was also limited by low AI credit availability from the education account. This introduces the balance to be made between ease of interface use, AI credit cost and accessibility of information.

The Copilot models through Microsoft 365 were more consistently able to provide compartmental model code in line with the academic papers of interest. This relative success was also mirrored through the LLMs' ability to fit the models to data and compute model comparison metrics. The outputs here were

not hindered by AI credit usage or issues with copying publicly accessible information. However, weaker versions of the Copilot models provided code inconsistent with the recommendations of the LLM with the intention of reducing time taken to produce output.

Further, during the fitting to data prompting, we noticed problems with the LLMs' choice of parameters for estimation. In some cases, the LLM attempted to estimate more parameters than could be reliably identified from the available data, and in other cases, too few parameters were estimated for the model to reproduce the observed behaviour. This highlights the need to check the model output and consider the validity of the fitted model.

The LLaMA vllm was used to generate compartmental model code and showed great resilience when aiming to learn new syntax. This often came at the cost of long response times where at times the model seemed stuck in loops. This open-source LLM was able to generate correct code using both Odin and PyGOM, as well as successfully transform basic models to suit an Ebola outbreak when given suitable prompting for style of parameterisation.

Despite the challenges outlined, the LLMs frequently provided useful insights into the mathematical modelling aspects of the project and considered the implications of epidemiological features specific to Ebola. Further prompting to rectify the issues often led to more appropriate code and improved formatting. Effective use of LLMs requires an understanding of the underlying compartmental models as well as the structure and purpose of the code. This enables the user to evaluate the output code and guide the LLM to an appropriate solution. LLMs can then be viewed as a tool to support these processes, and hopefully still contribute to producing efficient and effective decisions in the face of disease outbreaks. Consequently, a key limitation to the use of LLMs in this context is differing accessibility due to AI and healthcare restrictions.

References:

1. UK Health Security Agency (2021). *UK Health Security Agency*. [online] GOV.UK. Available at: <https://www.gov.uk/government/organisations/uk-health-security-agency> [Accessed 5 Aug. 2026].
2. Gao, S., Chakraborty, A.K., Greiner, R., Lewis, M.A. and Wang, H. (2025). Early detection of disease outbreaks and non-outbreaks using incidence data: A framework using feature-based time series classification and machine learning. *PLOS Computational Biology*, 21(2), p.e1012782. doi:10.1371/journal.pcbi.1012782.
3. Shandross, L., Howerton, E., Contamin, L., Hochheiser, H., Krystalli, A., Consortium of Infectious Disease Modeling Hubs, Reich, N.G., Ray, E.L., Castro Rivadeneira, A.J., Contamin, L., Funk, S., Gerding, A., Gruson, H., Hochheiser, H., Howerton, E., Kerr, M., Krystalli, A., Loo, S.L., Ray, E.L., Reich, N.G., Sato, K., Shandross, L., Sherratt, K., Truelove, S. and Zorn, M. (2026). Multi-Model Ensembles in Infectious Disease and Public Health: Methods, Interpretation, and Implementation in R. *Statistics in Medicine*, [online] 45(1–2), p.e70333. doi:10.1002/sim.70333.
4. Matthew James Keeling and Rohani, P. (2008). *Modeling infectious diseases in humans and animals*. Princeton: Princeton University Press.
5. LEGRAND, J., GRAIS, R.F., BOELLE, P.Y., VALLERON, A.J. and FLAHAULT, A. (2006). Understanding the dynamics of Ebola epidemics. *Epidemiology and Infection*, [online] 135(4), pp.610–621. doi:10.1017/s0950268806007217.
6. World Health Organization (2025). *Ebola Disease*. [online] *Who.int*. World Health Organization: WHO. Available at: <https://www.who.int/news-room/fact-sheets/detail/ebola-disease> [Accessed 5 Aug. 2026].
7. Github.io. (2026). 1 *Getting started with odin – Odin and Monty*. [online] Available at: <https://mrc-ide.github.io/odin-monty/odin.html> [Accessed 5 Aug. 2026].
8. Readthedocs.io. (2019). *Welcome to the documentation of pygom — PyGOM Documentation 0.1.7.dev61+g5c10d9c documentation*. [online] Available at: <https://pygom.readthedocs.io/en/master/> [Accessed 5 Aug. 2026].
9. Tye, E., Finnie, T., Hall, I. and Leach, S. (2018). *PyGOM - A Python Package for Simplifying Modelling with Systems of Ordinary Differential Equations*. [online] arXiv.org. Available at: <https://arxiv.org/abs/1803.06934> [Accessed 6 Aug. 2026].
10. Martcheva, M. (2015). Fitting Models to Data. *Texts in Applied Mathematics*, pp.123–148. doi:10.1007/978-1-4899-7612-3_6.

11. Hoang, M.T. and Ehrhardt, M. (2025). Differential equation models for infectious diseases: Mathematical modeling, qualitative analysis, numerical methods and applications. *SeMA Journal*. doi:10.1007/s40324-025-00404-9.
12. Domenech de Celles, M. and Kramer, S. (2026). A comparison of observation models for statistical inference of emerging disease transmission dynamics: Application to SARS-CoV-2. *MedRxiv Journal*. doi: <https://doi.org/10.64898/2026.01.27.26344924>.
13. Tang, L., Zhou, Y., Wang, L., Purkayastha, S., Zhang, L., He, J., Wang, F. and Song, P.X.K. (2020). A Review of Multi-Compartment Infectious Disease Models. *International Statistical Review*, [online] 88(2), pp.462–513. doi:10.1111/insr.12402.
14. Generable.com. (2018). *Optimizing, Sampling, and Choosing Priors* | Arya Pourzanjani | Generable. [online] Available at: <https://www.generable.com/post/optimizing-sampling-and-choosing-priors> [Accessed 5 Aug. 2026].
15. Marwan Shams Eddin, Hajj, H. El, Ramez Zayyat and Lee, G. (2025). Systematic Comparison of Different Compartmental Models for Predicting COVID-19 Progression. *PubMed*, [online] 6(3). doi:10.3390/epidemiologia6030033.
16. Bates, S.S., Hastie, T. and Tibshirani, R. (2021). Cross-validation: what does it estimate and how well does it do it? *Journal of the American Statistical Association*, pp.1–22. doi:10.1080/01621459.2023.2197686.
17. INRB-UMIE (2026). *GitHub - INRB-UMIE/BDBV2026-Data: Data and scripts for epidemiological analysis of the 2026 Bundibugyo Ebola outbreak*. [online] GitHub. Available at: <https://github.com/INRB-UMIE/BDBV2026-Data/tree/main> [Accessed 5 Aug. 2026].
18. Nash, R.K., Bhatia, S., Morgenstern, C., Doohan, P., Jorgensen, D., McCain, K., McCabe, R., Nikitin, D., Forna, A., Cuomo-Dannenburg, G., Hicks, J.T., Sheppard, R.J., Naidoo, T., Sabine van Elsland, Geismar, C., Rawson, T., Sequoia Iris Leuba, Wardle, J., Routledge, I., Fraser, K., Natsuko Imai-Eaton, Cori, A. and Unwin, J.T. (2024). Ebola virus disease mathematical models and epidemiological parameters: a systematic review. *The Lancet Infectious Diseases*. [online] doi:10.1016/s1473-3099(24)00374-8.

19. D'Silva, J.P. and Eisenberg, M.C. (2017). Modeling spatial invasion of Ebola in West Africa. *Journal of Theoretical Biology*, 428, pp.65–75.
doi:10.1016/j.jtbi.2017.05.034.
20. Santermans, E., Robesyn, E., Ganyani, T., Sudre, B., Faes, C., Quinten, C., Van Bortel, W., Haber, T., Kovac, T., Van Reeth, F., Testa, M., Hens, N. and Plachouras, D. (2016). Spatiotemporal Evolution of Ebola Virus Disease at Sub-National Level during the 2014 West Africa Epidemic: Model Scrutiny and Data Meagreness. *PLOS ONE*, [online] 11(1), p.e0147172.
doi:10.1371/journal.pone.0147172.
21. Do, T.S. and Lee, Y.S. (2016). Modeling the Spread of Ebola. *Osong Public Health and Research Perspectives*, 7(1), pp.43–48.
doi:10.1016/j.phrp.2015.12.012.
22. Berge, T., Lubuma, J.M.-S., Moremedi, G.M., Morris, N. and Kondera-Shava, R. (2016). A simple mathematical model for Ebola in Africa. *Journal of Biological Dynamics*, 11(1), pp.42–74. doi:10.1080/17513758.2016.1229817.
23. Grinsztajn, L., Semenova, E., Margossian, C.C. and Riou, J. (2021). Bayesian workflow for disease transmission modeling in Stan. *Statistics in Medicine*, 40(27), pp.6209–6234. doi:10.1002/sim.9164.